Contents

1 Chapter 04 — Game Abstraction & Scaling Imperfect-Information Games: Implementation Report **1**
1.1 1. What was developed 1
1.2 2. Abstraction modules 2
1.3 3. CFR+ timed benchmark 3
1.4 4. Pareto frontier 6
1.5 5. Cross-validation 7
1.6 6. Key learnings 7
1.7 7. Reproduction 8

1 Chapter 04 — Game Abstraction & Scaling Imperfect-Information Games: Implementation Report

Environment: May 2026

Games: Fixed-limit Leduc, Mini-NL Leduc, Extended Leduc

Algorithms: Vanilla CFR, CFR+, MCCFR, information/action abstraction

Targets: Lossless abstraction preserves Nash quality; lossy abstraction exposes an exploitability floor; abstraction-size vs exploitability Pareto frontier produced

Status: Core targets achieved

1.1 1. What was developed

Chapter 04 extends the Chapter 03 CFR toolkit with explicit game abstraction. The implementation builds a Leduc-family pipeline that can shrink the game along three axes, solve the abstracted game, translate the resulting strategy back into the full game, and measure the exploitability cost.

Source layout:

```
implementation/step04/
├── exploration/
│   ├── leduc_suit_abstracted.py       # early suit-isomorphism probe
│   ├── leduc_lossy_abstracted.py      # early lossy rank-bucket probe
│   ├── day01_*                        # CFR/MCCFR comparison scripts
│   ├── day02/mini_nl_leduc.py         # early variable-bet game
│   └── figures/                       # exploration figures
├── phase4/
│   ├── leduc_full_engine.py           # full fixed-limit Leduc baseline
│   ├── leduc_rank_engine.py           # rank-canonical / suit-isomorphic Leduc
│   ├── mini_nl_leduc.py               # variable-bet Mini-NL Leduc
```

```
|   ├── extended_leduc.py              # 4-rank, 2-suit Leduc variant
|   ├── cfr_trainer.py                 # generic vanilla CFR trainer
|   ├── exploitability.py              # full-game exploitability evaluator
|   ├── day01_*                        # lossless suit-isomorphism experiments
|   ├── day02_*                        # card bucketing, HSD, EMD, k-means
|   ├── day03_*                        # action abstraction and translators
|   ├── day04_*                        # combined abstraction pipeline
|   ├── day05_*                        # exploitability gap and Pareto frontier
|   ├── day06_openspiel_compare.py     # OpenSpiel cross-validation
|   └── day07_cfrplus_panels.py        # 180 s CFR+ benchmark panels
└── EXPERIMENT_RESULTS.md              # final CFR+ result summary
```

1.2 2. Abstraction modules

1.2.1 2.1 Lossless suit isomorphism

Leduc suits carry no strategic information because payoffs depend only on rank and whether the private card pairs the community card. Chapter 04 therefore implements rank-canonical engines that merge suit-isomorphic information sets.

This abstraction is lossless: it should preserve Nash quality while reducing the number of information sets and increasing the number of CFR/CFR+ iterations possible under the same wall-clock budget.

1.2.2 2.2 Lossy card bucketing

The lossy abstraction path computes hand-strength features, compares bucket candidates with EMD-style distances, and clusters information sets with configurable bucket counts.

Two recall regimes are represented:

Regime	Meaning
Perfect-recall buckets	Bucket current private/public card strength but retain the previous bucket trail
Imperfect-recall buckets	Replace part of the earlier history with bucket identity, shrinking the tree more aggressively

The goal is not to make this abstraction exact. The goal is to measure the exploitability floor created by merging strategically distinct states.

1.2.3 2.3 Action abstraction and translation

Mini-NL Leduc introduces variable bet sizes. The action abstraction then restricts the action set and evaluates how the abstract strategy performs when deployed in the fuller action game.

Implemented translators:

Translator	Role
Nearest-action	Map an off-grid bet to the closest abstract bet
Probability-split	Split mass between neighboring abstract bets
Pseudo-harmonic	Poker-specific odds-space interpolation; implemented for comparison and future extension

1.2.4 2.4 Extended and combined abstraction

Extended Leduc increases the benchmark to four ranks and two suits. The combined pipeline composes suit isomorphism, action abstraction, and card bucketing, then evaluates the resulting strategy in the appropriate full game family.

1.3 3. CFR+ timed benchmark

The final benchmark uses CFR+ with regret flooring and linear strategy averaging.

Setting	Value
Training budget	180 seconds per configuration
Repetitions	3 runs, seeds 1, 2, 3
Metric	Final exploitability after training
Raw outputs	`phase4/.day07_cfrplus_results.json`, `phase4/.day07_cfrplus_results.csv`

1.3.1 3.1 Fixed-limit Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full CFR+	936	4.44e-05	2,655
Suit iso	288	3.13e-06	14,185
k2 perfect	132	0.571	11,399
k3 perfect	228	0.382	11,043

Configuration	Info sets	Mean exploitability	Mean iterations
full bucket perfect	288	4.18e-06	10,806

The lossless suit-isomorphic abstraction is the best fixed-limit result. It reduces storage and reaches lower exploitability than full CFR+ under the same wall-clock budget because it completes many more iterations. Full-bucket variants behave similarly because, once rank/post-flop cases are separated, the remaining compression is effectively suit isomorphism.

Lossy coarse buckets remain highly exploitable. Additional CFR+ iterations do not close that gap because the abstraction itself has removed strategically relevant information.

1.3.2 3.2 Mini-NL Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full mini-NL	4,704	0.00692	439
Action abstraction	936	0.673	2,338

The action-abstracted game trains many more iterations because the tree is smaller, but the final exploitability remains high. This is the strongest evidence in the step that action abstraction and deployment translation can dominate the total error.

1.3.3 3.3 Extended Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full extended	10,304	0.0272	111
Suit iso	2,968	0.00126	671
Suit + action	504	4.696	4,145
Suit + action + buckets	108	4.734	3,694

Extended Leduc confirms the value of lossless abstraction at a larger scale: suit isomorphism cuts the information-set count by about 71% and reaches much lower exploitability under the same wall-clock budget. The action-abstracted configurations are useful as failure cases: aggressive action compression creates compact games, but the translated strategies are highly exploitable in the fuller action game.

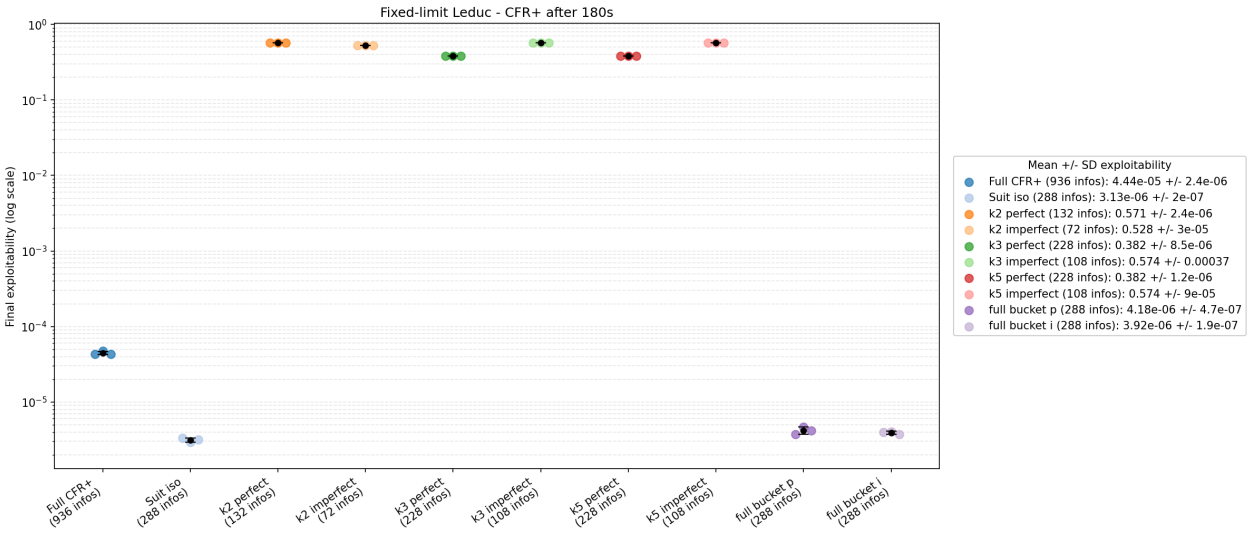

Figure 1: Fixed-limit Leduc CFR+ abstraction results

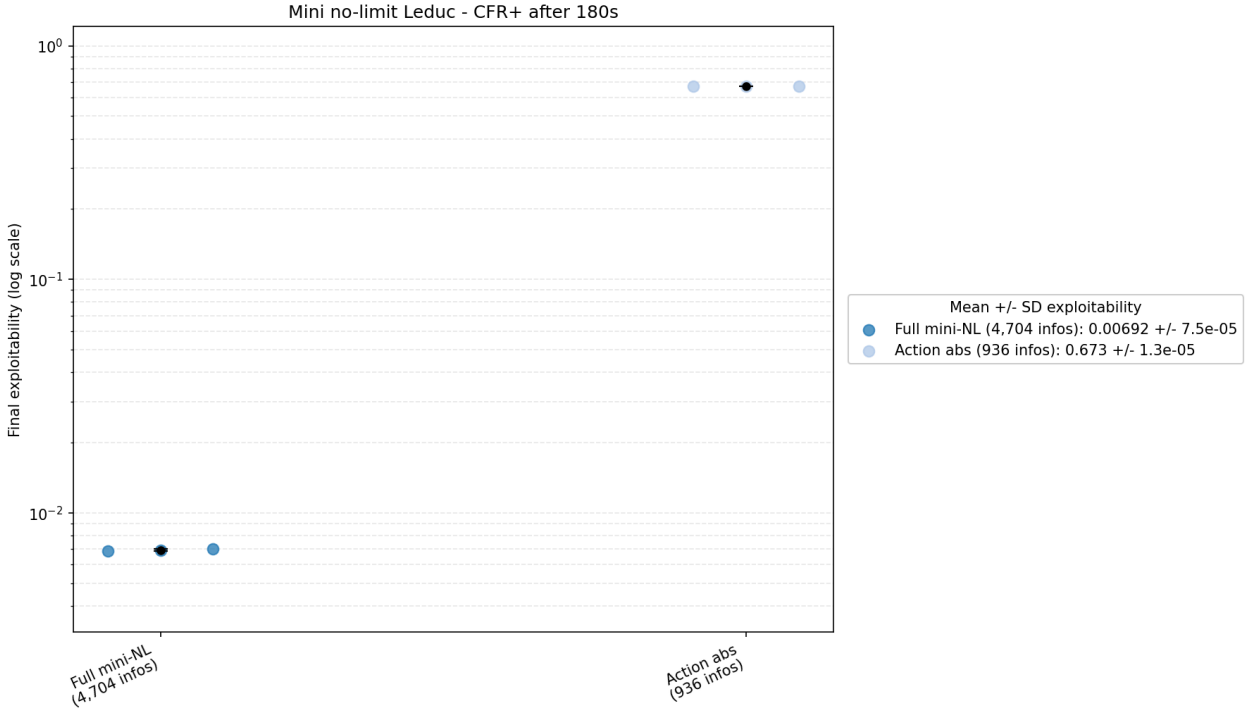

Figure 2: Mini-NL Leduc CFR+ abstraction results

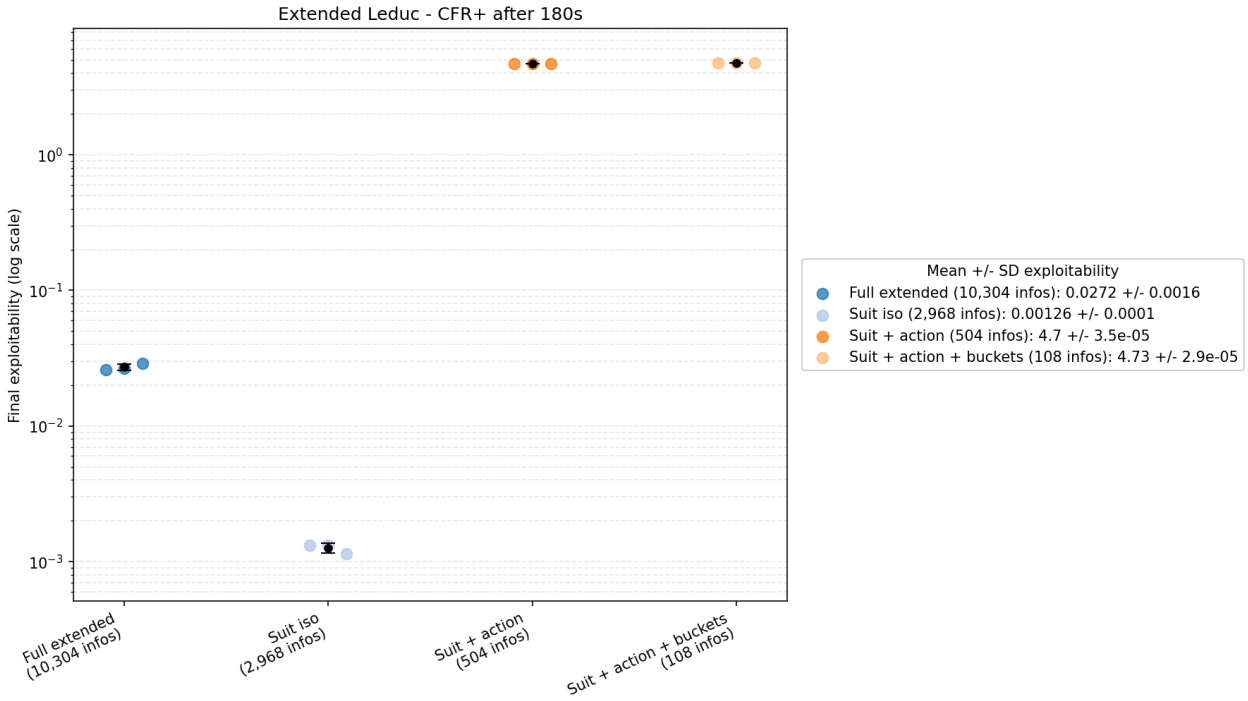

Figure 3: Extended Leduc CFR+ abstraction results

1.4 4. Pareto frontier

The Pareto frontier summarizes the central Chapter 04 tradeoff: smaller abstract games train faster, but not every reduction is strategically safe.

Abstraction family	Result
Suit isomorphism	Best tradeoff; smaller game with no strategic loss in Leduc
Full rank buckets	Behaves close to suit isomorphism
Coarse lossy buckets	Smaller trees, persistent exploitability floor
Action abstraction	Largest risk; translation error dominates in Mini-NL and Extended Leduc

The practical criterion is therefore not just "smaller is better." A useful abstraction must improve compute enough to compensate for the exploitability gap it introduces.

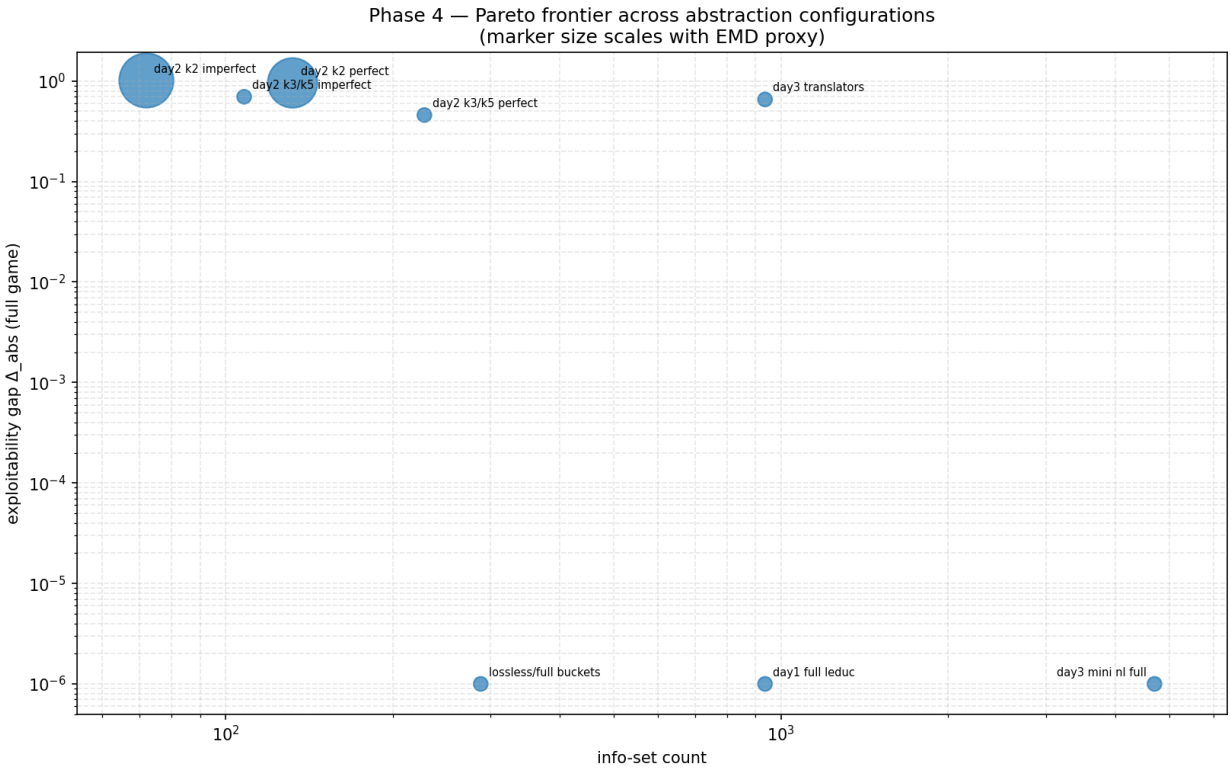

Figure 4: Abstraction Pareto frontier

1.5 5. Cross-validation

`day06_openspiel_compare.py` aligns all 936 Leduc information sets against OpenSpiel and verifies that the custom implementation is in the same strategic regime. The remaining differences are interpreted as update-convention differences rather than exact trajectory mismatches, so this is a sanity check rather than a proof of identical solver dynamics.

1.6 6. Key learnings

1. **Lossless abstraction is the highest-value compression.** In Leduc-family games, suit isomorphism reduces the game substantially and improves wall-clock convergence without introducing abstraction error.
2. **Lossy information abstraction creates an exploitability floor.** Coarse card buckets solve faster but cannot recover distinctions they deliberately removed.
3. **Action abstraction is more dangerous than card abstraction in these experiments.** Restricted action sets plus translation produced high exploitability even when the abstract tree was much smaller.
4. **The Pareto frontier is the right reporting format.** Strategy quality must be presented

together with game size and abstraction type.

5. **Subgame solving is the natural next mechanism.** Static translation is brittle; Chapter 6's safe/nested solving methods are the production-grade response to off-tree actions.

1.7 7. Reproduction

```
# From repo root:
cd implementation/step04/phase4

# Smoke-test budget used by the phase-4 integration notes:
python day01_train.py --iterations 200
python day02_train.py --iterations 200
python day03_train.py --iterations 100
python day04_train.py --iterations 20000
python day05_train.py
python day06_openspiel_compare.py --iterations 200

# Final CFR+ benchmark panels:
python day07_cfrplus_panels.py
```

The generated JSON/CSV files are written into `implementation/step04/phase4/`, and figures are written into `implementation/step04/phase4/figures/`.